

Exercício A2

Arquitetura CI/CD com Jenkins

Gustavo Champam

Componente: DevOps & QA

Relatório de execução das três práticas - 05/10/2026

Repositório e rastreabilidade

<https://github.com/Gustavo-Champam/atividade-a2-jenkins-cicd>

Execução remota: [GitHub Actions - laboratório A2](#)

Commit executado: 24ca6ff6ff5e186461d386f050d51f48c9d79509

Síntese dos resultados

Prática 1: Freestyle com falha controlada (5 testes, 2 falhas) e correção (5 aprovados). JUnit e JaCoCo publicados. Cobertura de linhas: 86,36%.

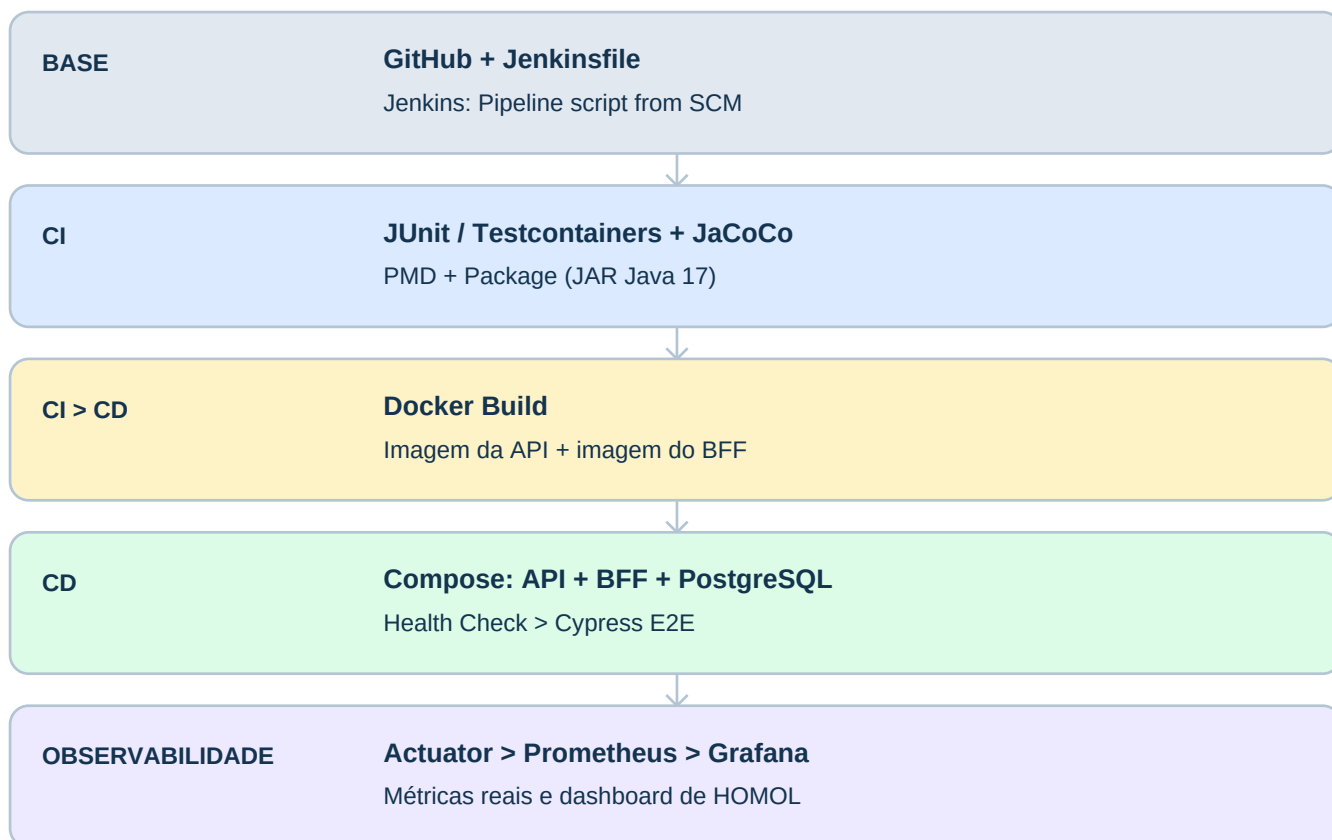
Práticas 2 e 3: Jenkinsfile único executado pelo Jenkins, resultado SUCCESS. 7 testes Java e 3 cenários Cypress aprovados; PostgreSQL real, deploy HOMOL, health UP, target Prometheus UP e dashboard Grafana com quatro painéis.

Adaptação do ambiente

A Prática 1 foi executada no Jenkins local em 29/09/2026. Para as Práticas 2 e 3, o Docker Engine foi disponibilizado em uma máquina Linux temporária do GitHub Actions, pois o Docker Desktop local não inicia. O Jenkins continua sendo o executor da esteira e carrega o Jenkinsfile via Git; Actions apenas hospeda o laboratório e recolhe os arquivos de evidência.

O projeto usado nas Práticas 2 e 3 é a versão portátil V5.1 disponibilizada pela própria professora, evolução do Jogo Enigma citado no enunciado. A aplicação e os testes de negócio foram preservados; as adaptações de infraestrutura estão versionadas no repositório. Não houve publicação em produção.

1. Arquitetura efetivamente executada



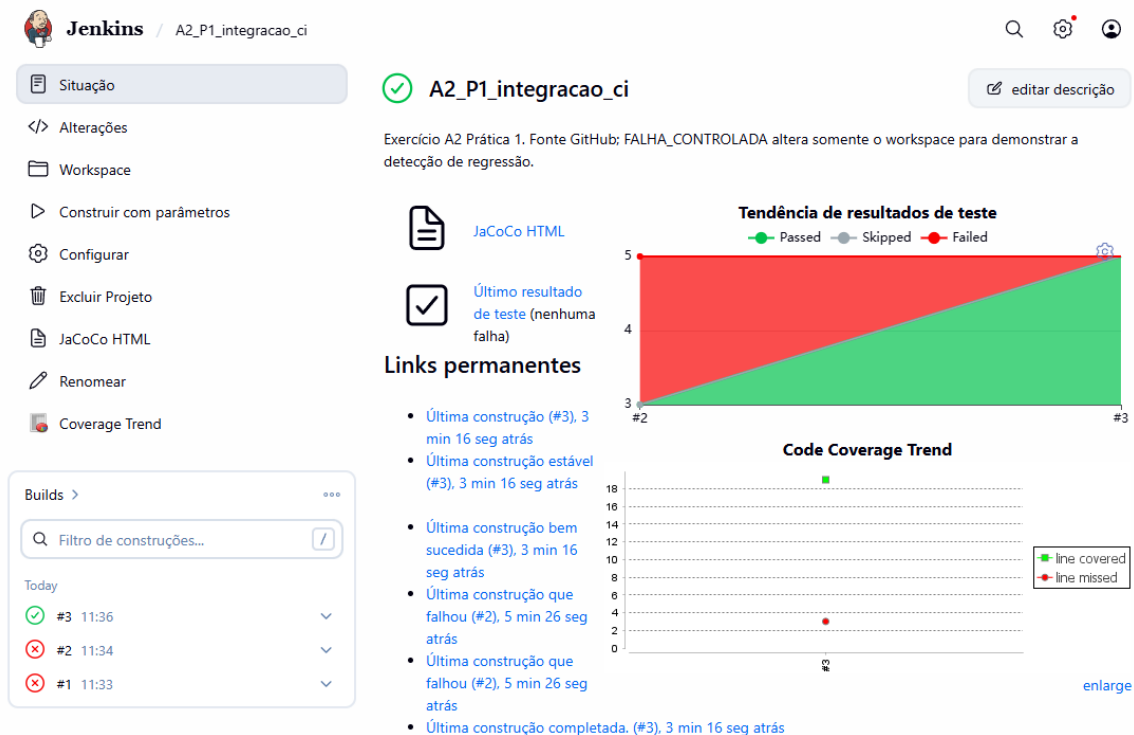
Fluxo de negócio validado: navegador/Cypress > BFF Node/Express > API Spring Boot > PostgreSQL. O teste de repository também sobe seu próprio PostgreSQL com Testcontainers antes da criação da imagem de homologação.

Serviço	Porta no host	Comunicação interna
Jenkins	9090	Orquestra Maven e Docker
API	8080	api:8080
BFF / Front	3000	bff:3000
PostgreSQL	Sem porta pública	postgres:5432
Prometheus	9091	prometheus:9090
Grafana	3001	grafana:3000

As URLs localhost das capturas pertencem ao runner Linux. O runner é descartado ao terminar; não são endpoints permanentes no computador do aluno. As portas publicadas ficam vinculadas ao endereço local do runner.

2. Prática 1 - Freestyle e CI

Job A2_P1_integracao_ci. SCM: https://github.com/adleles/integracao_v1; commit de origem 07ea28faf22037c3d314c37c4c429078c127756b. Execução Maven clean verify com JDK 17; publicadores JUnit e JaCoCo configurados.



Em qual etapa ocorre CI?

Na obtenção do código pelo Git e execução automatizada da compilação, testes e verificações. O build do laboratório foi disparado manualmente; a automação interna é feita pelo Jenkins. Um webhook pode automatizar o disparo a cada alteração.

Por que Jenkins não substitui Maven, JUnit e JaCoCo?

Jenkins coordena e registra as etapas. Maven resolve dependências e executa o ciclo de build. JUnit executa testes. JaCoCo mede o código exercitado. São responsabilidades complementares.

Plugins usados na prática: Git, Maven Integration, JUnit e JaCoCo. O padrão de resultados JUnit é `**/target/surefire-reports/*.xml`. A configuração XML do job acompanha o pacote de evidências.

3. Experimento - falha controlada e diagnóstico

O parâmetro FALHA_CONTROLADA altera no workspace o limite de frequência de 75 para 76. Assim, dois casos que esperavam aprovação com frequência de 75% passam a falhar. O build nº 2 terminou em FAILURE: 5 testes, 2 falhas, 0 erros.

Jenkins / A2_P1_integracao_ci / #2

Estado pessoal

Alterações

Saída do console

Editar informações de compilação

Apagar a construção (0)

Parâmetros

Git Build Data

Resultado de testes

Construção anterior

Próxima construção

#2 (29 de set. de 2026 11:34:01)

Adicionar descrição

Deixar essa construção como permanente

Iniciado pelo(a) usuário(a) **gustavo-a2**

Iniciado 5 min 27 seg atrás
Levou 9,9 seg

Revision: 07ea28faf22037c3d314c37c4c429078c127756b
Repository: https://github.com/adleles/integracao_v1.git

refs/remotes/origin/master

Resultado de testes (2 falhas)

integracao.AlunoTest.testAprovacao_AprovacaoFinal
integracao.AlunoTest.testAprovacao_AprovacaoNota

Sem mudanças.

REST API Jenkins 2.541.3

```
[INFO]
[ERROR] Failures:
[ERROR] AlunoTest.testAprovacao_AprovacaoFinal:46 expected: <true> but was: <false>
[ERROR] AlunoTest.testAprovacao_AprovacaoNota:35 expected: <true> but was: <false>
[INFO]
[ERROR] Tests run: 5, Failures: 2, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD FAILURE
[INFO] -----
[INFO] Total time: 2.644 s
[INFO] Finished at: 2026-09-29T11:34:11-03:00
[INFO] -----
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin:3.5.4:test (default-test) on project integracao_v1: There are test failures.
[ERROR]
[ERROR] See C:\Users\Gustavo\Documents\Codex\2026-09-29\https-github-com-adleles-integracao-v1\work\jenkins\home\workspace\A2_P1_integracao_ci\target\surefire-reports
for the individual test results.
[ERROR] See dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and
[date].dumpstream.
[ERROR] -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the
following articles:
```

O que o build vermelho informa antes do deploy?

Uma condição verificada pela esteira falhou. Aqui a regra de negócio foi quebrada e os testes detectaram o defeito. O console permite distinguir falha funcional de falha de infraestrutura. A promoção do artefato deve ser interrompida até a correção.

4. Correção - build verde e JUnit publicado

A correção desativa FALHA_CONTROLADA. CleanBeforeCheckout restaura a implementação original do Git. O build nº 3 concluiu em SUCCESS: 5 testes aprovados, sem falhas, erros ou testes ignorados. Nenhum defeito foi publicado no repositório de origem.

Estado pessoal

Alterações

Saída do console

Editar informações de compilação

Apagar a construção {0}

Parâmetros

Git Build Data

Resultado de testes

Coverage Report

JaCoCo HTML

Construção anterior

✓ #3 (29 de set. de 2026 11:36:11)

Adicionar descrição

Deixar essa construção como permanente

Iniciado pelo(a) usuário(a) [gustavo-a2](#)

Iniciado 3 min 19 seg atrás
Levou 9 seg

git

Revision: 07ea28faf22037c3d314c37c4c429078c127756b
Repository: https://github.com/adleles/integracao_v1.git

• refs/remotes/origin/master

✓

Resultado de testes (nenhuma falha)

Jacoco - Overall Coverage Summary

INSTRUCTION	87%	
BRANCH	100%	
COMPLEXITY	77%	
LINE	86%	
METHOD	67%	
CLASS	100%	

</>

Sem mudanças.

REST API

Jenkins 2.541.3

Estado pessoal

Alterações

Saída do console

Editar informações de compilação

Apagar a construção {0}

Parâmetros

Git Build Data

Resultado de testes

Coverage Report

JaCoCo HTML

Construção anterior

Resultado de teste 5

✓ 5

↗ 2

Levou 68 ms.

All tests are passing!
Nice one! All 5 tests are passing.

Todos os testes

Pacote	Falha	Pular	Passed	Total	Duração
integracao	0 -2	0	5 +2	5	35 ms

REST API

Jenkins 2.541.3

5. JaCoCo e comparação das evidências

Jenkins

A2_P1_integracao_ci

#3

Jacoco

Estado pessoal

Alterações

Saída do console

Editar informações de compilação

Apagar a construção {0}

Parâmetros

Git Build Data

Resultado de testes

Coverage Report

JaCoCo HTML

Construção anterior

Download jacoco.exec binary coverage file

18

16

14

12

10

8

6

4

2

0

line covered

line missed

Overall Coverage Summary

name	instruction	branch	complexity	line	method	class
all classes	87% M: 9 C: 58	100% M: 0 C: 8	77% M: 3 C: 10	86% M: 3 C: 19	67% M: 3 C: 6	100% M: 0 C: 1

Coverage Breakdown by Package

name	instruction	branch	complexity	line	method	class
integracao	87% M: 9 C: 58	100% M: 0 C: 8	77% M: 3 C: 10	86% M: 3 C: 19	67% M: 3 C: 6	100% M: 0 C: 1

REST API Jenkins 2.541.3

Verificação	Build 2 - falha	Build 3 - correção
Resultado	FAILURE	SUCCESS
JUnit	5 testes / 2 falhas	5 testes / 0 falhas
JaCoCo publicado	Falha em test impede a fase verify	Linhas 86,36%; ramos 100,00%

Qual a vantagem de publicar JUnit e JaCoCo no job?

As falhas, detalhes de cada teste e cobertura ficam junto ao histórico dos builds. Isso facilita diagnóstico e comparação entre versões. Cobertura indica código exercitado; não prova ausência de defeitos nem qualidade suficiente das asserções.

O primeiro build do histórico foi uma tentativa de configuração com erro de dependência de plugin. Ele não representa a falha controlada. As evidências do experimento são os builds 2 e 3, identificados acima.

6. Prática 2 - Stage View do pipeline único

Jenkins / A2_P2_P3_Jogo_Enigma

Status

Changes

Build Now

Delete Pipeline

Configure

Rename

Full Stage View

JaCoCo

Pipeline Syntax

A2_P2_P3 Jogo_Enigma

Praticas 2 e 3: Jenkinsfile do GitHub; JUnit, JaCoCo, PMD, Docker, Cypress e observabilidade. Ambiente temporario GitHub Actions.

Last Successful Artifacts

Test Result Trend

Passed Skipped Failed

#1

Stage View

BUILDS

Filter

Today

#1 4:00 PM

Average stage times:
(full run time: ~4min 42s)

#1

Oct 05
16:00

No Changes

Checkout	JUnit + JaCoCo	PMD	Package	Docker Build	Deploy HOMOL	Health Check	Cypress E2E	Observabilidade	Declarative Post Actions
780ms	39s	11s	5s	49s	20s	15s	56s	1min 21s	95ms

Latest Test Result (no failures)

Permalinks

- Last build (#1), 4 min 47 sec ago
- Last stable build (#1), 4 min 47 sec ago
- Last successful build (#1), 4 min 47 sec ago
- Last completed build (#1), 4 min 47 sec ago

REST API jenkins 2.580.1

Captura real do Jenkins remoto. As Práticas 2 e 3 usam o mesmo job e o mesmo Jenkinsfile. O resultado e a duração de cada stage também estão preservados em stages.json.

7. Pipeline as Code e classificação dos stages

O job A2_P2_P3_Jogo_Enigma é do tipo Pipeline. Sua configuração usa CpsScmFlowDefinition (Pipeline script from SCM), Git, branch main e Script Path = Jenkinsfile. O arquivo jenkins-job.xml e o console preservam a configuração e o commit realmente obtido.

Stage	Grupo	Responsabilidade
Checkout	Base	Obtém a versão do Git e identifica o commit
JUnit + JaCoCo	CI	Testes Java, PostgreSQL de teste e cobertura
PMD	CI	Análise estática e relatório
Package	CI	Geração do JAR
Docker Build	CI > CD	Criação das imagens da API e BFF
Deploy HOMOL	CD	Disponibiliza os serviços com Compose
Health Check	CD	Espera e exige resposta UP da API
Cypress E2E	CD	Valida o sistema integrado já implantado
Observabilidade	Operação / QA	Verifica coleta e quatro consultas de métricas

Qual a diferença entre Freestyle e Jenkinsfile?

Freestyle guarda etapas e publicadores na configuração manual do job. Pipeline as Code guarda a sequência, condições e tratamento dos resultados em um arquivo Git. O Jenkins lê essa definição a cada execução.

Por que versionar o pipeline junto ao código?

A definição pode ser revisada, comparada e revertida junto à aplicação. O commit associa o código às regras de build e deploy usadas, melhorando rastreabilidade e reprodução.

Se os testes falharem, container e deploy devem continuar?

Não. Neste Jenkinsfile, o retorno não zero do Maven interrompe o fluxo normal e os próximos stages são ignorados. Ações post/always preservam relatórios para diagnóstico. Não há catchError mascarando uma falha funcional.

8. Resultados Java, cobertura e análise estática

Suíte Java	Testes	Falhas	Erros	Ignorados
EmailTest	2	0	0	0
ParticipanteControllerTest	1	0	0	0
ParticipanteRepositoryTest	1	0	0	0
ParticipanteRequestTest	1	0	0	0
ParticipanteServiceTest	1	0	0	0
ParticipanteTest	1	0	0	0

Foram executados 7 testes Java. O teste de repository usa PostgreSQL 17 real com Testcontainers, sem substituição por H2. Os demais verificam Entity, VO, DTO, Service e Controller.

JaCoCo da API: cobertura de linhas 83,33%; ramos 50,00%. Cobertura de instruções: 48,48%. O relatório completo acompanha os arquivos de evidência.

The screenshot shows the Jenkins CI/CD interface for a build named 'A2_P2_P3_Jogo_Enigma'. The left sidebar contains navigation links: Status, Changes, Console Output, Delete build '#1', Git Build Data, Tests (selected), JaCoCo, Edit build information, Restart from Stage, Replay, Pipeline Steps, and Workspaces. The main area displays 'Tests 10' with a green checkmark and 'Took 19 sec'. Below this is a large blue box with confetti and the text 'All tests are passing! Nice one! All 10 tests are passing.' Underneath, there is a table titled 'All Tests' showing the results for various packages.

Package	Failed	Skipped	Passed	Total	Duration
(root)	0	0	3 +3	3 +3	1.3 sec
br.edu.exemplo.jogoenigma.controller	0	0	1 +1	1 +1	0.15 sec
br.edu.exemplo.jogoenigma.dto	0	0	1 +1	1 +1	87 ms
br.edu.exemplo.jogoenigma.entity	0	0	1 +1	1 +1	2 ms
br.edu.exemplo.jogoenigma.repository	0	0	1 +1	1 +1	0.27 sec
br.edu.exemplo.jogoenigma.service	0	0	1 +1	1 +1	1.6 sec
br.edu.exemplo.jogoenigma.vo	0	0	2 +2	2 +2	44 ms

REST API Jenkins 2.580.1

PMD foi executado pelo objetivo `pmd:pmd` e seu relatório foi arquivado. Não se executa `pmd:check`; a configuração didática original também mantém `failOnViolation=false`. A análise registra os alertas sem aplicar um bloqueio por violações. Isso não significa que o código tenha zero alertas de qualidade.

9. Prática 3 - containers e health check

Container / serviço	Estado	Imagem
api	running	jogo-enigma-api:1
bff	running	a2homol-bff
grafana	running	grafana/grafana:12.1.1
pgadmin	running	dpage/pgadmin4:9
postgres	running healthy	postgres:17-alpine
prometheus	running	prom/prometheus:v3.5.0

Transcrição estruturada de `docker compose ps --format json`, coletada no runner após o deploy. O arquivo bruto `containers.json` e o console do Jenkins preservam a evidência; esta tabela não é uma captura simulada de terminal.

Health check observado: `{"status": "UP", "groups": ["liveness", "readiness"]}`. O pipeline exige HTTP válido e status UP com tentativas limitadas.

```
Pretty print
{"status": "UP", "groups": ["liveness", "readiness"]}
```

Por que BUILD SUCCESS não garante operação futura?

Ele confirma apenas as verificações configuradas naquele instante. Banco, rede, configuração ou recursos podem falhar depois. O Health Check verifica prontidão após o deploy, enquanto métricas ajudam a acompanhar o comportamento ao longo do tempo.

Qual é o papel do Docker Compose?

Descrever e iniciar os serviços de homologação, rede, volumes e configurações. O banco precisa estar saudável antes da API. A ordem de inicialização, sozinha, não comprova disponibilidade; por isso há uma verificação explícita depois do deploy.

10. Frontend e Cypress ponta a ponta

Jogo Enigma

Front → BFF Node.js → API Spring Boot → PostgreSQL

Nome	E-mail	0	Cadastrar
-----------------------------------	-------------------------------------	--------------------------------	--

Participantes

- Ana E2E - ana1791216241190@teste.com - 80 - MESTRE
- Dup - dup1791216242628@teste.com - 10 - INICIANTE

Frontend servido pelo BFF em 3000. Cypress executou os três cenários originais: abrir a home, cadastrar participante ponta a ponta e rejeitar e-mail duplicado. As capturas individuais, vídeos e XMLs do Cypress acompanham a entrega.

11. Cypress - evidência do cadastro

Jogo Enigma

Front → BFF Node.js → API Spring Boot → PostgreSQL

Nome	E-mail	0	Cadastrar
-----------------------------------	-------------------------------------	--------------------------------	--

Cadastrado com sucesso

Participantes

- Ana E2E - ana1791216241190@teste.com - 80 - MESTRE

Por que JUnit antes e Cypress depois do deploy?

JUnit fornece feedback rápido sobre as unidades e integrações controladas antes de promover a imagem.

Cypress precisa do ambiente pronto para verificar o fluxo navegador, BFF, API e PostgreSQL. Ele detecta erros de integração e configuração que testes isolados não alcançam.

Resultado E2E: 3 cenários aprovados, 0 falhas. A segunda captura registra o cadastro de Ana E2E e a classificação MESTRE. O terceiro cenário envia o mesmo e-mail duas vezes e exige HTTP 400 na tentativa duplicada.

12. Prometheus - target UP

Prometheus

Query

Alerts

Status > Target health

Select scrape pool

Filter by target health

Filter by endpoint or labels

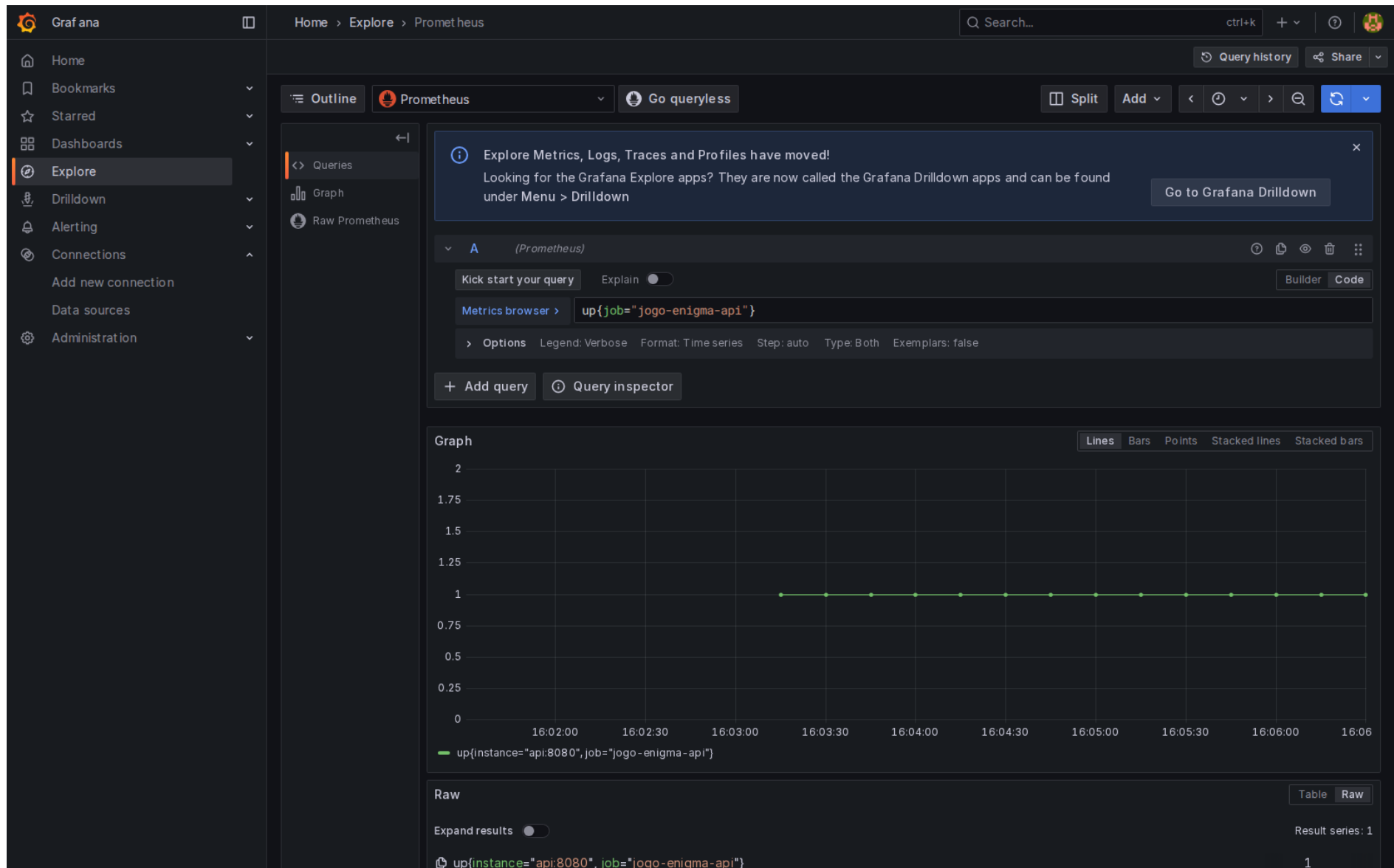
jogo-enigma-api

1 / 1 up

Endpoint	Labels	Last scrape	State
http://api:8080/actuator/prometheus	instance="api:8080" job="jogo-enigma-api"	6.962s ago 10ms	UP

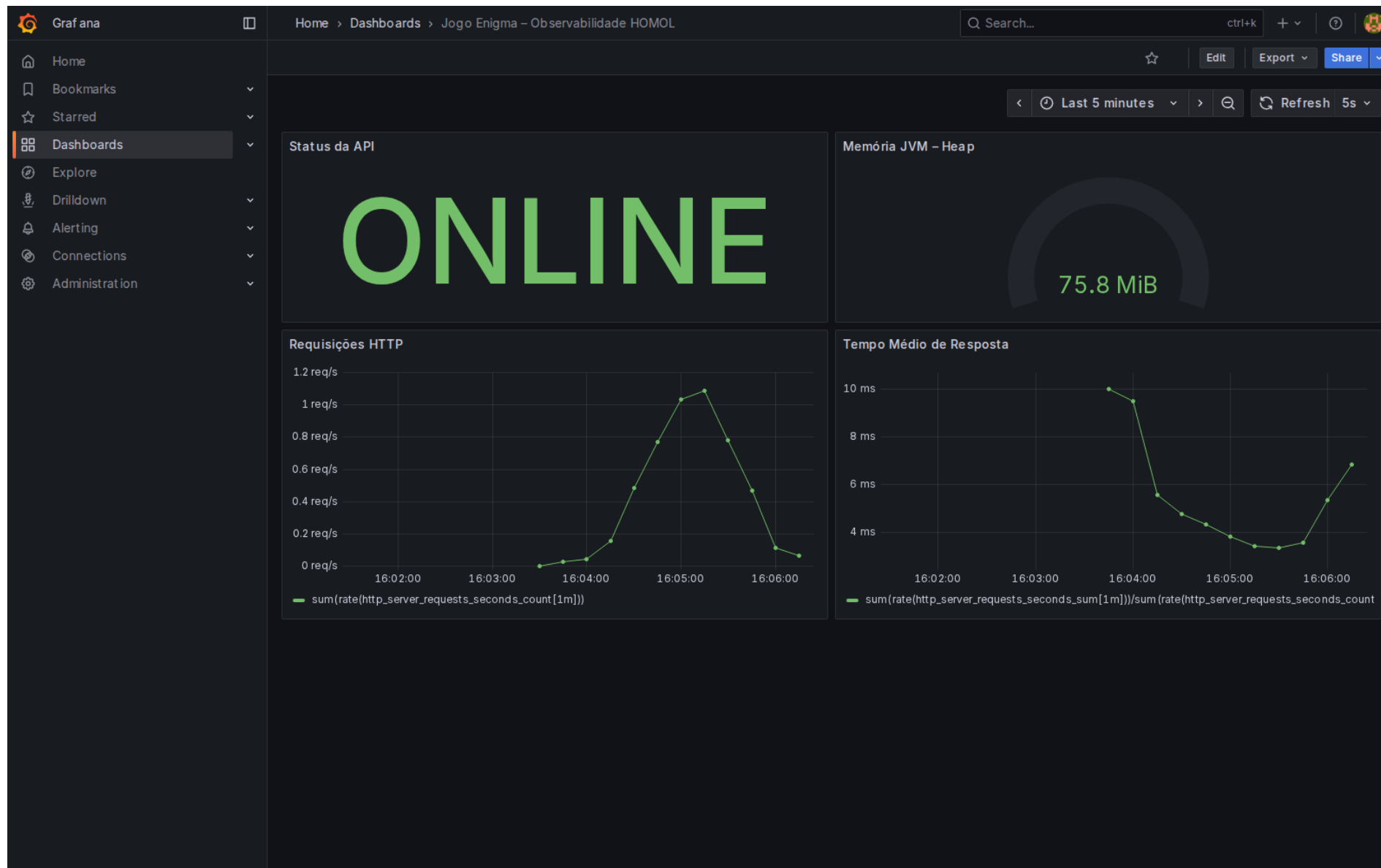
Target jogo-enigma-api, endpoint <http://api:8080/actuator/prometheus>. A porta externa é 9091:9090, evitando conflito com o Jenkins em 9090. O arquivo targets.json confirma o estado health=up.

13. Grafana - consulta up e datasource



Consulta real no Explore: `up{job='jogo-enigma-api'}`, resultado 1. Esse valor indica sucesso na última coleta do Prometheus; não prova sozinho que todos os fluxos de negócio estão corretos. O datasource usa `http://prometheus:9090` na rede Docker.

14. Dashboard solicitado - quatro painéis



Jogo Enigma - Observabilidade HOMOL. Dashboard provisionado e validado com métricas reais. As requisições usadas para gerar tráfego são verificações repetidas da API/BFF. O ambiente é temporário, sem alegação de monitoramento contínuo após o término do runner.

15. Consultas e interpretação da observabilidade

Status da API

```
up{job="jogo-enigma-api"}
```

Valor coletado na verificação: 1.

Requisições HTTP

```
sum(rate(http_server_requests_seconds_count[1m]))
```

Valor coletado na verificação: 1.08881630113548.

Tempo Médio de Resposta

```
sum(rate(http_server_requests_seconds_sum[1m]))/sum(rate(http_server_requests_seconds_count[1m]))
```

Valor coletado na verificação: 0.0034391373061224487.

Memória JVM - Heap

```
sum(jvm_memory_used_bytes{area="heap"})
```

Valor coletado na verificação: 77386000.

Status usa Stat com 1 = ONLINE e 0 = OFFLINE. Requisições e tempo médio usam Time series; tempo é apresentado em segundos. Heap usa Gauge com unidade bytes IEC. As consultas seguem o tutorial e pressupõem o laboratório com uma aplicação monitorada.

Qual a relação entre Prometheus e Grafana?

Spring Boot/Micrometer expõe /actuator/prometheus. Prometheus coleta periodicamente esse endpoint e armazena séries temporais. Grafana consulta essas séries via PromQL e desenha indicadores e gráficos; não coleta diretamente a aplicação.

Como observabilidade se relaciona com qualidade?

Ela permite observar disponibilidade da coleta, tráfego, latência e consumo de memória depois do deploy. Complementa os testes ao revelar degradações operacionais e orientar a investigação, sem substituir testes de comportamento.

16. Fechamento e conclusão arquitetural

1. Onde termina CI e começa CD?

Checkout estabelece a base. Compilação, JUnit, JaCoCo, PMD e Package verificam e produzem o artefato. Docker Build é a ponte entre CI e CD; Deploy HOMOL marca a disponibilização efetiva da aplicação. Health Check e Cypress validam o resultado dessa implantação.

2. Qual vantagem arquitetural oferece o Jenkinsfile?

Torna a orquestração parte versionada e revisável da arquitetura. A definição acompanha o código, reduz divergências manuais e registra a ordem das dependências entre verificações e deploy.

3. Por que JUnit antes e Cypress depois?

JUnit impede a promoção de código que já viola comportamentos conhecidos. Cypress depende dos serviços implantados para testar o fluxo completo, incluindo comunicação e persistência.

4. Como a observabilidade amplia a qualidade?

Acrescenta evidências do comportamento operacional depois da construção e entrega. Testes validam cenários previstos; métricas ajudam a identificar degradações durante a execução.

5. O que foi reutilizado da A1?

JUnit, JaCoCo e PMD continuam sendo mecanismos de QA. Na A2, essas ferramentas passam a executar dentro de uma esteira Jenkins rastreável, antes da construção e implantação dos containers.

6. Dashboard conforme o tutorial

O dashboard contém os quatro painéis, consultas, tipos de visualização e unidade de heap solicitados. Sua captura real está na seção 14 e a definição JSON acompanha o pacote.

O que mudou arquiteturalmente de A1 para A2?

A1 concentra-se nos mecanismos de qualidade do código. A2 integra esses mecanismos a um processo versionado que obtém o código, verifica, empacota, cria containers, implanta em homologação e valida o sistema integrado. Prometheus e Grafana estendem a análise de qualidade ao comportamento após o deploy. Jenkins coordena esse fluxo, preservando o papel específico de cada ferramenta.

17. Conferência da entrega e fontes

Exigência	Evidência neste PDF
Prática 1 (25%)	Seções 2 a 5: Freestyle, falha/correção, JUnit e JaCoCo
Prática 2 (30%)	Seções 6 a 8: Stage View, SCM, stages e resultados
Prática 3 (30%)	Seções 9 a 15: containers, health, E2E, target UP, Explore e dashboard
Análise arquitetural (15%)	Seções 1, 7, 15 e 16: fluxo e respostas
Git e rastreabilidade	Capa, commit, execução Actions e repositório abaixo

Fontes da atividade

Exercicio_A2_Arquitetura_CI_CD_Jenkins_ATUALIZADO.docx - enunciado fornecido.

Pratica_1 (Ac2)_Jenkins_Projeto_Integracao_CI_Junit_Jacoco_Plugins.pptx - roteiro Freestyle.

Tutorial_Dashboard_Grafana_A2.docx - painéis e consultas.

[Projeto integracao_v1 da professora](#)

[Jogo Enigma V5.1 da professora](#)

[Repositório da atividade](#)

[Execução remota com logs e artefatos](#)

Arquivos de apoio e limites da comprovação

O ZIP complementar contém código, Jenkinsfile, Compose, dashboard e os arquivos originais de evidência. O PDF é o documento único solicitado para entrega. As capturas registram execução real; o desenho de arquitetura e as tabelas são explicações/transcrições identificadas.

Adaptações: host Linux remoto em lugar do Docker Desktop; versão portátil V5.1 da própria professora; senhas temporárias por execução em lugar de admin/admin; dashboard provisionado por arquivo. Mantiveram-se os componentes, portas, estágios, consultas e objetivos do enunciado. Não há Docker Hub nem produção, pois não são exigidos nesta atividade.

O laboratório é acadêmico e temporário. Jenkins mantém os resultados durante a execução; o artefato Actions é retido por 30 dias. O ZIP local preserva as evidências independentemente dessa retenção.